REWARD-TOKEN TAIL AUDITS FOR TRAJECTORY TRANSFORMER PLANNING

Anonymous authors
Paper under double-blind review

ABSTRACT

Trajectory Transformers plan by decoding interleaved state, action, and reward tokens. This paper studies a failure mode that is specific to that interface: a score-only planner can select strings whose reward-token sums improve while the selected prefixes become unlikely under the offline data distribution and the decoded next-state tokens stop matching the decoded actions. The selected object is therefore not merely an action sequence; it is a full trajectory-token string with support, prefix, and dynamics structure. We build a reproducible autoregressive surrogate that keeps this planning interface while making every token probability auditable. Across four controls, candidate-count stress, horizon sweeps, context-length ablations, tokenizer-resolution stress, sampling-temperature stress, tail-bias stress, sieve ablations, exact finite-candidate laws, and a Gymnasium Pendulum-v1 benchmark (Towers et al., 2024), the experiments separate reward-token extremization from realized simulator utility. In the expanded candidate-count stress, increasing the score-only candidate count from 1 to 256 raises decoded reward-token return by 5.093 while lowering realized simulator return by 2.532; the Support-Calibrated Plan Sieve improves realized return over score-only selection by 8.822 at the same candidate count. On Pendulum-v1, increasing raw candidate count from 1 to 64 raises decoded reward-token return by 28.875 while true return changes by -0.660; the selected high-candidate plans remain 9.663 return units below a behavior-policy baseline, and a support-gated behavior fallback repairs that gap while reducing token/simulator mismatch by 1.491. The claims are deliberately bounded: the evidence supports a mechanism and benchmark stress test, not a benchmark leaderboard result.

1 INTRODUCTION

Offline reinforcement learning must plan under a distributional constraint: the data can contain useful behavior without covering every high-scoring action sequence that a learned model might imagine. Trajectory Transformers (TTs) make this constraint unusually concrete. A TT does not only output actions. It models a complete sequence of state, action, and reward tokens and then plans by searching over decoded continuations (Janner et al., 2021). That design creates an attractive planning primitive: sample or decode many candidate trajectories, score the reward-token sums, and execute the action sequence from the highest-scoring candidate. It also creates a failure surface that is easy to miss if evaluation records only predicted return.

This paper asks a narrow question. When a TT-style planner increases the number of decoded candidates, can score-only selection push reward-token sums upward while selecting trajectory-token strings that are low-support, prefix-surprising, or dynamically inconsistent? The answer matters because many practical planning procedures treat candidate count as a simple inference budget. If more candidates merely explore useful plans, increasing the budget should help or at least leave realized utility unchanged. If candidate count also increases the chance of selecting a pathological high-score tail, then inference-time scaling can hurt even when the model appears to be producing higher reward-token scores.

We call the mechanism *reward-token extremization*. It is related to reward-model overoptimization (Gao et al., 2023), but it is not the same claim. In a TT, the optimized score is embedded inside the same token string that carries state and action predictions. A decoded trajectory can have reward

tokens that look excellent while its prefixes are improbable under the learned trajectory distribution or its state tokens disagree with the rollout implied by its action tokens. A generic reward-hacking diagnostic would notice the reward/utility gap; it would not explain whether the selected string is a coherent TT continuation.

The paper therefore treats the full token string as the unit of audit. Given a decoded candidate

$$(s_0, a_0, r_0, s_1, a_1, r_1, \ldots, s_{H-1}, a_{H-1}, r_{H-1}),$$

we record the decoded reward-token sum, the realized return obtained by executing the decoded action sequence in the simulator, the average token log-likelihood, the maximum prefix surprise, the token/simulator dynamics error, candidate diversity, selected-mode collapse, and a calibrated candidate-set risk rate. These diagnostics let us separate three explanations that are often conflated: the planner may be finding genuinely better actions, it may be selecting reward-token strings that still lie inside support, or it may be moving into a high-score low-support tail.

The implementation is intentionally small and inspectable. It is not a neural benchmark TT, and it does not claim benchmark-level offline RL performance. Instead, it is a controlled stress harness. The surrogate model is a smoothed autoregressive model over discretized state, action, and reward tokens. This surrogate preserves the TT planning interface while allowing exact measurement of token probabilities and prefix surprises. That design choice is a strength for mechanism discovery and a limitation for field-level benchmarking. The paper keeps those two statements together throughout.

The main empirical result is that reward-token extremization is measurable, repairable in the controlled setting, and sensitive to architecture-specific details. In the original four-control experiment, the horizon-stress setting shows candidate count 64 increasing raw predicted reward-token return by 5.572 while decreasing realized simulator return by 7.553. In the expanded suite, the high-candidate stress extends to 256 candidates: raw predicted reward rises by 5.093 from one candidate, raw realized return falls by 2.532, and the Support-Calibrated Plan Sieve improves realized return over raw score selection by 8.822. The method does not dominate every ablation slice; in a separate N=64 ablation, a likelihood-only filter obtains higher realized return than the full sieve, while the full sieve more strongly reduces prefix surprise and dynamics error. We report this boundary because a useful audit should make methods look worse when their assumptions are stressed.

The contribution is fivefold. First, we define a TT-specific audit for reward-token tails that measures prefix surprise and token/simulator consistency rather than only predicted reward. Second, we prove and test an exact finite-candidate selected-utility identity, clarifying when more score-selected candidates can lower expected realized utility. Third, we implement the Support-Calibrated Plan Sieve, a calibrated filter and penalty rule that uses offline support diagnostics to gate candidate sets. Fourth, we run a substantially expanded experiment suite with candidate-count, horizon, context, tokenizer, temperature, tail-bias, and ablation stress tests. Fifth, we add a standard Gymnasium Pendulum-v1 benchmark tier with a behavior-policy baseline, oracle upper bound, support-gated fallback, and generated claim gates, while keeping the claim narrower than benchmark leaderboard performance.

2 Problem Setting

2.1 Trajectory-token planning

Let a horizon-H trajectory-token string be

$$x_{1:3H} = (s_0, a_0, r_0, \ldots, s_{H-1}, a_{H-1}, r_{H-1}).$$

The TT planning interface models the string autoregressively,

$$p_\theta(x_{1:3H}) = \prod_{t=1}^{3H} p_\theta(x_t \mid x_{<t}),$$

and uses the decoded reward tokens to score candidate plans. In a neural TT, the conditional probabilities come from a Transformer. In this paper, they come from an inspectable smoothed context-count model. The important shared structure is the object being decoded: the planner searches over strings containing state, action, and reward tokens, not only over action sequences.

The raw score of a candidate string is the decoded reward-token return

$$S(x) = \sum_{t=0}^{H-1} \hat{r}_t.$$

The realized utility is measured separately:

$$U(x) = \sum_{t=0}^{H-1} r(s_t^{\text{sim}}, a_t),$$

where the simulator starts from the same initial state and executes the decoded actions. This distinction is central. The reward tokens are model predictions embedded in the string; the realized utility is what the decoded actions actually obtain under the environment.

2.2 FAILURE MODE

Score-only candidate selection chooses

$$\hat{x}_K \in \arg \max_{x \in \{X_1, \ldots, X_K\}} S(x),$$

where K is the candidate count. Increasing K changes the selected distribution even when the candidate generator is unchanged. If high reward-token strings are also high-utility strings, this is helpful. If high reward-token strings live in low-support regions or encode dynamics-inconsistent state transitions, the selected realized utility can fall.

In a TT this failure has three linked symptoms. The first is reward-token extremization: $S(\hat{x}_K)$ increases with K. The second is support drift: the selected string has low average log-likelihood or a surprising prefix under the offline trajectory distribution. The third is token-level dynamics mismatch: the next-state tokens in the selected string disagree with the next state implied by executing the decoded action. The failure is most convincing when all three symptoms appear while realized utility decreases.

2.3 AUDITED QUANTITIES

For a candidate x, the audit records:

$$\ell(x) = \frac{1}{3H} \sum_{t=1}^{3H} \log p_\theta(x_t \mid x_{<t}),$$

$$P(x) = \max_{1 \le t \le 3H} -\log p_\theta(x_t \mid x_{<t}),$$

and

$$D(x) = \frac{1}{H-1} \sum_{t=0}^{H-2} \left| \hat{s}_{t+1} - f(\hat{s}_t, \hat{a}_t) \right|.$$

Here $\ell(x)$ is average token log-likelihood, $P(x)$ is maximum prefix surprise, and $D(x)$ is token/simulator dynamics error. The candidate-set risk rate is the fraction of generated candidates that violate one or more calibrated support thresholds. Candidate diversity is the number of distinct mode signatures normalized by K. Selected-mode collapse records whether repeated candidate generation produces the same selected mode across evaluation episodes.

These quantities are not interchangeable. Average log-likelihood can miss a single catastrophic prefix. Prefix surprise can flag a rare transition even if the full string has ordinary average likelihood. Dynamics error can catch strings whose state tokens are plausible locally but not coherent under the decoded actions. The paper reports all three because the reviewer attack "this is only likelihood regularization" is otherwise too easy to make.

3 FINITE-CANDIDATE SELECTION LAW

Before discussing the TT-specific repair, we state the finite-candidate law used as a sanity check. It is intentionally generic and deliberately limited. It does not assume a TT and does not prove that every beam search implementation has the same behavior. It only characterizes score-selected utility for i.i.d. candidates from a finite distribution.

Proposition 1 (Exact score-selected utility identity)**.** *Let candidates $X_1, \dots, X_K$ be i.i.d. from a finite distribution over outcomes i with probability p_i, proxy score s_i, and realized utility u_i. Select a maximum-score candidate and break ties uniformly among tied samples. Define*

$$p_{=i} = \Pr(s(X) = s_i), \qquad p_{<i} = \Pr(s(X) < s_i).$$

Then

$$\Pr(\hat{X}_K = i) = K p_i \sum_{m=0}^{K-1} \binom{K-1}{m} \frac{p_{=i}^m p_{<i}^{K-1-m}}{m+1}.$$

Consequently,

$$\mathbb{E}[U(\hat{X}_K)] = \sum_i u_i \Pr(\hat{X}_K = i).$$

The proof distinguishes one sampled copy of outcome i. It is selected when no other sample has strictly higher score. If m other samples tie its score, uniform tie-breaking selects the distinguished copy with probability $1/(m+1)$. The remaining $K-1-m$ samples must have lower score. Multiplying by the K possible positions gives the expression.

The proposition has a simple implication. If a high-score tail has lower realized utility than the lower-score mass, and the probability of selecting that tail rises with candidate count, expected realized utility can decrease as K increases. This is not a universal anti-scaling theorem. It is a diagnostic identity: the sign of inference-time scaling depends on the joint distribution of proxy score and realized utility. A well-aligned high-score tail helps. A misaligned high-score tail hurts.

The experiments use this identity in two ways. First, a closed-form tail-misalignment distribution verifies that the implementation agrees with Monte Carlo. Second, the identity gives language for the TT experiments: candidate count is not merely a computational budget; it changes which part of the decoded trajectory distribution becomes selected.

4 SUPPORT-CALIBRATED PLAN SIEVE

4.1 CALIBRATION

The Support-Calibrated Plan Sieve is a deployable diagnostic repair for the controlled setting. It calibrates thresholds from offline training trajectories. Let τ_ℓ be a low quantile of average log-likelihood, τ_P be a high quantile of prefix surprise, and τ_D be a high quantile of dynamics error. A candidate is considered support-compatible when

$$\ell(x) \geq \tau_\ell, \qquad P(x) \leq \tau_P, \qquad D(x) \leq \tau_D.$$

The method also calibrates a candidate-set risk ceiling τ_ρ. If the fraction of candidates violating the thresholds exceeds this ceiling, the planner marks the set as unsafe for further candidate-count expansion. This flag is important because a repair that silently returns a plan can hide the fact that the generator is mostly producing out-of-support strings.

4.2 SELECTION

Accepted candidates are ranked by the penalized score

$$S_{\text{sieve}}(x) = S(x) - \lambda_\ell[\tau_\ell - \ell(x)]_+ - \lambda_P[P(x) - \tau_P]_+ - \lambda_D[D(x) - \tau_D]_+.$$

If at least one candidate passes all thresholds, the method returns the accepted candidate with highest penalized score. If no candidate passes, it returns the lowest-risk candidate and marks the episode as blocked. The blocked flag is not a failure of bookkeeping; it is part of the method's output. In an offline planning system, "do not trust this expanded candidate set" can be safer than pretending every search budget yields a valid plan.

4.3 WHY THIS IS NOT ONLY LIKELIHOOD REGULARIZATION

Likelihood regularization is one component of the method, but the audit is broader. A token string can have acceptable average likelihood while containing one surprising prefix. It can also have plausible local tokens while the decoded next-state sequence fails to match the decoded actions. The full method therefore tests likelihood, prefix surprise, and dynamics consistency separately. The ablation suite confirms that these components trade off: a likelihood-only filter gives the highest realized return in one N=64 ablation slice, while the full sieve more strongly reduces the two token-pathology metrics that motivate the audit. This is a useful boundary, not an embarrassment. It says the method is a conservative diagnostic gate, not an oracle.

5 EXPERIMENTAL DESIGN

5.1 ENVIRONMENT AND DATA

The environment is a one-dimensional control problem with a support-limited high-return corridor. Offline behavior mostly visits moderate-return actions, while trajectories near the high-return corridor are rare and noisy. This setup is intentionally chosen to make the difference between decoded reward-token score and realized utility observable. It is not a benchmark suite and does not support claims about average offline RL performance.

The offline dataset is generated once per experiment configuration. The autoregressive model is trained from discretized state, action, and reward tokens. During evaluation, the candidate generator samples trajectory strings from the fitted model at a fixed initial condition. Raw selection chooses the highest decoded reward-token sum. The oracle selector chooses the highest realized utility and is reported only as an upper-bound diagnostic. The Support-Calibrated Plan Sieve applies calibrated support gates before ranking candidates.

5.2 PRIMARY CONTROLS

The primary run evaluates four controls:

- **In-support**: candidate strings are encouraged to remain near common offline behavior.
- **Out-of-support**: the generator is placed under a stress distribution with less support overlap.
- **Anti-aligned scorer**: reward-token scores are deliberately easier to increase in a way that can misalign with simulator utility.
- **Horizon stress**: the horizon is long enough for prefix and dynamics errors to compound.

The primary run uses candidate counts $\{1, 2, 4, 8, 16, 32, 64\}$, 640 training trajectories, 24 evaluation episodes, and seed 13. The expanded suite uses smaller episode batches per cell to keep memory and runtime controlled while adding many more stress factors.

5.3 EXPANDED STRESS SUITE

The expanded suite adds eight families of tests:

- Candidate-count tail stress up to $K = 256$.
- Horizon sweep over base horizons 6, 10, 16, and 22.
- Autoregressive context-length sweep over contexts 2, 5, and 8.
- Tokenizer-resolution sweep over coarse, default, and fine discretizations.
- Sampling-temperature sweep over temperatures 0.85, 1.12, 1.45, and 1.80.
- Sieve ablation comparing raw selection, likelihood-only, prefix-only, dynamics-only, full, lenient, and strict gates.
- Reward/action tail-bias stress.

- Exact finite-candidate law stress for harmful and benign high-score tails.

All grids run sequentially and write compact CSV/JSON outputs to disk. The suite does not keep large candidate tensors in memory after aggregation. This matters for reproducibility: the goal is stronger evidence without turning a small controlled audit into a RAM-heavy benchmark job.

5.4 Gymnasium Pendulum benchmark

The v4 benchmark tier uses Gymnasium Pendulum-v1 (Towers et al., 2024). This is not a leaderboard experiment: the model is still the inspectable autoregressive trajectory-token surrogate, and the question is whether the reward-token tail pathology survives in a standard continuous-control environment. Pendulum-v1 is useful because its state, action, and reward are continuous, its true dynamics are deterministic and exactly replayable, and a behavior-policy baseline is easy to define without training a second algorithm.

The benchmark trains on 280 offline behavior trajectories of horizon 16. The behavior mixture is mostly medium-gain stabilizing control, with a smaller expert component and a rare high-torque component. The evaluation uses 16 fixed initial states and nested candidate pools with $K \in \{1, 8, 32, 64\}$, so every larger candidate count contains the smaller candidate set for the same seed. Raw selection chooses the largest decoded reward-token sum. The support-gated fallback either selects an accepted candidate or abstains to a deterministic behavior-policy rollout when the candidate set is entirely unsupported. The oracle chooses the highest true return in the sampled pool and is reported only as an analysis upper bound. All results are written to `results/pendulum_benchmark/`.

5.5 Acceptance criteria

The paper's claim audit requires that every headline claim be traceable to an output file. The expanded audit passes only if the generated claims satisfy numerical thresholds for reward-token extremization, realized-return degradation, high-candidate repair, horizon sensitivity, context sensitivity, tokenizer sensitivity, temperature sensitivity, tail-bias sensitivity, and finite-law behavior. The Pendulum audit adds gates for benchmark reward-token extremization, lack of true-return improvement, behavior-fallback repair, dynamics-mismatch reduction, behavior-baseline underperformance, and oracle gap. When a proposed claim fails, the claim is rewritten or removed. For example, the first full run did not support the claim that temperature substantially changes candidate support-risk rate; it did support the more precise claim that temperature changes high-candidate realized return while risk rates remain saturated. The final manuscript uses the latter claim.

6 Main Results

6.1 Four-control summary

Table 1 summarizes the primary four-control run. The horizon-stress case is the clearest failure: raw selection at $K = 64$ raises predicted reward-token return by 5.572 relative to $K = 1$, but realized simulator return changes by -7.553. The sieve improves realized return over raw selection by 5.455 at $K = 64$. The out-of-support control is different: predicted return rises and realized return also rises, although the sieve still improves over raw. This heterogeneity is important. The paper does not claim that candidate-count scaling always hurts. It claims that candidate-count scaling can expose misaligned reward-token tails, and that the TT-specific audit can detect when it does.

6.2 Candidate-count tail stress to 256

The expanded candidate-count stress extends the horizon-stress setting to $K = 256$. Table 2 shows the key pattern. Raw predicted reward-token return rises from 24.339 at $K = 1$ to 29.433 at $K = 256$, a gain of 5.093. Raw realized return falls from 16.217 to 13.684, a change of -2.532. At $K = 256$, the sieve returns 22.507 realized return, improving over raw by 8.822. The oracle reaches 28.611, showing that good candidates are present but score-only reward-token selection does not reliably choose them.

Table 1: Primary four-control run from `results/summary.json`. Predicted and realized changes compare raw score-only selection at $K = 64$ to $K = 1$. The repair column is sieve minus raw realized return at $K = 64$.

Control	Pred. change	Realized change	Sieve repair	Risk at 64
In support	3.188	-0.969	2.002	0.725
Out of support	4.233	2.941	1.673	1.000
Anti-aligned scorer	2.870	-1.392	2.664	0.992
Horizon stress	5.572	-7.553	5.455	0.989

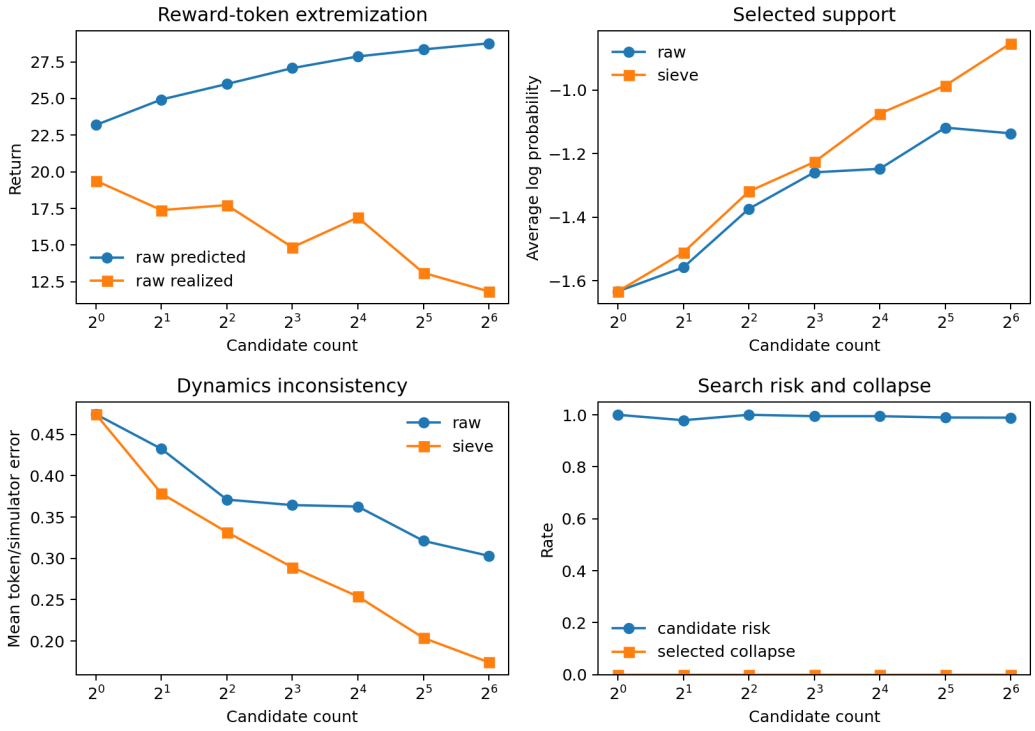

Figure 1: Primary horizon-stress control. Raw score-only selection raises decoded reward-token return as candidate count grows, while selected support and dynamics consistency deteriorate.

The $K = 64$ row is a useful warning against overselling the repair. In the expanded suite, the sieve is worse than raw at $K = 64$ for realized return, even though it reduces dynamics error. At $K = 128$ and $K = 256$, it strongly improves realized return. This is why the paper states the repair claim at the high-candidate tail where the audit passes, and why the ablation section discusses non-monotone tradeoffs. A method that only reports its winning slice would be easier to market and weaker scientifically.

6.3 Gymnasium Pendulum-v1 benchmark

Table 3 and Figure 4 give the standard-environment benchmark tier. The benchmark uses the exact Pendulum-v1 dynamics and reward (Towers et al., 2024), but keeps the TT-style object of study: the planner samples full state/action/reward token strings from the autoregressive surrogate and selects by decoded reward-token return. The result is not a leaderboard claim. It is a benchmark stress test for the same token-level failure mode.

The high-candidate raw selector is strongly reward-extremizing: from $K = 1$ to $K = 64$, decoded reward-token return rises by 28.875. True return does not follow; the mean true-return change is

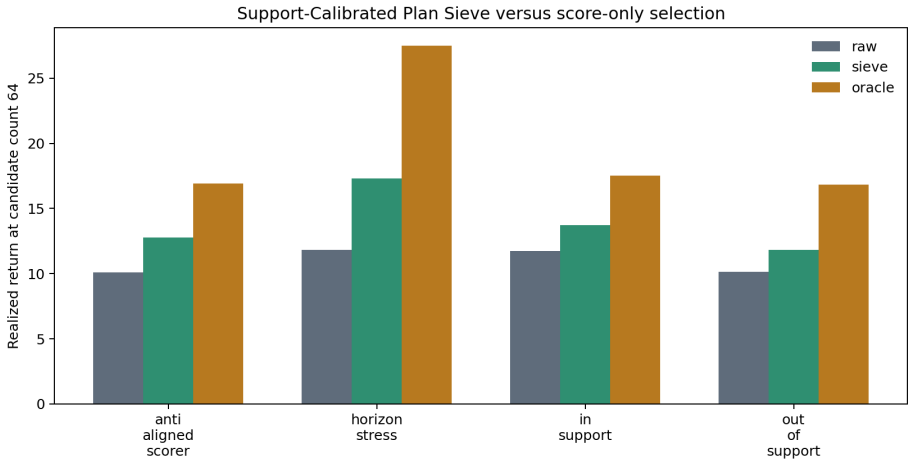

Figure 2: Realized return at $K = 64$ in the primary run. The oracle is a diagnostic upper bound, not a deployable method. The deployable comparison is raw score-only selection versus the Support-Calibrated Plan Sieve.

Table 2: Expanded candidate-count stress from the expansion metrics. The setting is horizon stress.

K	Method	Pred. return	Realized return	Prefix surprise	Dynamics error
1	Raw	24.339	16.217	5.867	0.393
16	Raw	27.902	22.509	5.298	0.373
64	Raw	28.930	15.502	5.166	0.334
128	Raw	29.122	12.888	5.011	0.321
256	Raw	29.433	13.684	6.754	0.334
64	Sieve	27.687	13.174	4.151	0.160
128	Sieve	27.400	22.689	3.497	0.167
256	Sieve	27.807	22.507	3.440	0.132
256	Oracle	24.674	28.611	5.485	0.393

-0.660, and at $K = 64$ raw selection is 9.663 return units below the behavior-policy baseline. The oracle gap of 9.090 shows that the sampled pool contains better action sequences than the raw reward-token selector chooses. Because every high-candidate raw set violates the calibrated support gates, the conservative repair abstains to the behavior-policy fallback. That fallback improves true return by 9.663 over raw and reduces token/simulator dynamics mismatch by 1.491.

6.4 Control panels

Figure 5 shows the primary controls across candidate counts. The in-support control is less fragile, while anti-aligned and horizon-stress variants expose the selection-risk mechanism. The out-of-support setting has high risk but can still improve realized return under raw selection, showing that high support risk is not by itself equivalent to low utility. The audit is therefore a conjunction: reward-token extremization, support/dynamics pathology, and realized utility movement must be read together.

7 Expanded Robustness Tests

7.1 Horizon sweep

The horizon sweep changes the base horizon while holding raw $K = 64$ selection fixed. Table 4 shows that longer horizons increase decoded reward-token scale: predicted return rises from 29.289

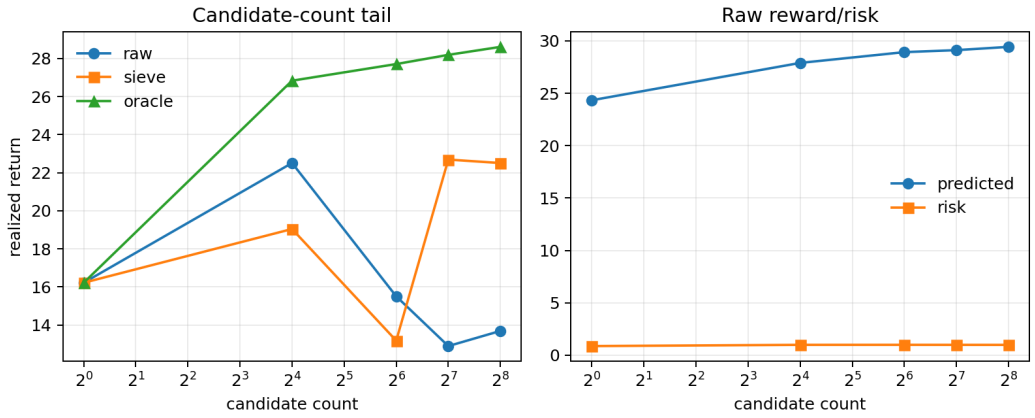

Figure 3: Expanded candidate-count stress. Raw reward-token scores increase with candidate count, but realized return is non-monotone and drops at high K. The sieve gives up some decoded reward-token score to recover realized utility at $K = 128$ and $K = 256$.

Table 3: Gymnasium Pendulum-v1 benchmark, effects at $K = 64$ over 16 fixed initial states. Values are means; brackets give bootstrap 95% intervals.

Effect	Mean [95% CI]
Raw reward-token gain, $K = 64$ minus $K = 1$	$28.875\ [22.253, 34.900]$
Raw true-return change, $K = 64$ minus $K = 1$	$-0.660\ [-5.773, 2.632]$
Behavior fallback minus raw true return at $K = 64$	$9.663\ [1.899, 18.372]$
Raw minus behavior-policy true return at $K = 64$	$-9.663\ [-18.372, -1.899]$
Raw minus fallback dynamics mismatch at $K = 64$	$1.491\ [1.365, 1.629]$
Oracle minus raw true return at $K = 64$	$9.090\ [4.694, 15.748]$

at horizon 6 to 51.250 at horizon 22. Realized return does not follow the same scale. It is 12.048 at horizon 6, 10.963 at horizon 10, 11.459 at horizon 16, and 23.827 at horizon 22. The result is not ”longer horizon always worse.” It is more specific: horizon changes how reward-token score, prefix surprise, and dynamics error compose, so candidate-count audits should be rerun when horizon changes.

7.2 Context Length

Context length controls how much autoregressive history is used by the token model. Table 5 shows that risk is sensitive to this choice. At context 2, the raw $K = 64$ candidate risk rate is 0.824. At context 5 it is 0.990, and at context 8 it reaches 1.000. The realized return changes from 8.177 to 10.868 and 10.146. Longer context can make rare prefixes sharper rather than safer; the effect depends on how the offline data covers long histories.

7.3 Tokenizer Resolution

Tokenizer resolution is a TT-specific design choice because the planner scores discrete state, action, and reward tokens. Table 6 shows that discretization changes the observed dynamics error. Coarse tokens give dynamics error 0.176 and realized return 15.958. Default tokens give dynamics error 0.222 and realized return 12.786. Fine tokens increase dynamics error to 0.482 while realized return rises to 18.363. The fine-token result is another example of why the audit should not collapse to one metric. A finer tokenizer can expose larger token/simulator inconsistency while still allowing some high-return action sequences.

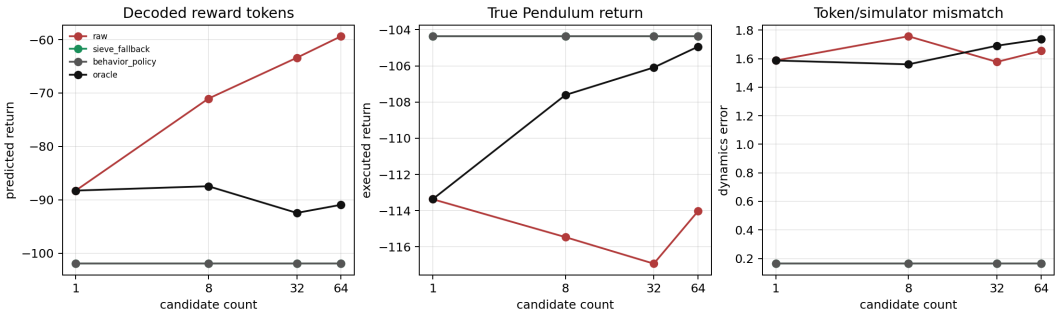

Figure 4: Gymnasium Pendulum-v1 benchmark. Raw reward-token selection drives decoded reward upward with candidate count, but true executed return stays below a behavior-policy baseline and far below the oracle candidate. The support-gated fallback abstains to behavior when the sampled token strings are unsupported.

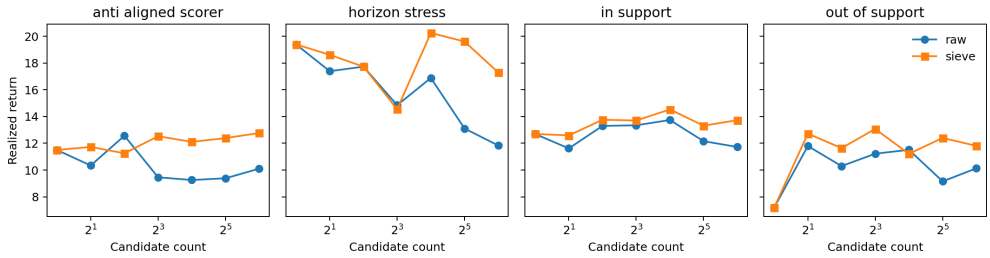

Figure 5: Primary controls across candidate counts. The result is not a universal monotonic failure. It is a conditional tail-selection failure that appears most sharply when reward tokens are easy to extremize in low-support regions.

7.4 Sampling temperature

Sampling temperature changes the candidate distribution. The first proposed claim for this experiment was that temperature would substantially change support-risk rate. The data did not support that claim: risk rates remain high and close together. The supported claim is more precise. Temperature changes high-candidate realized return while support-risk rates are saturated. Table 7 shows realized return ranging from 15.229 to 20.294 across the four temperatures, a span of 5.065, while risk rates stay between 0.980 and 0.992.

7.5 Reward and action tail bias

The tail-bias stress changes how often the data-generating process exposes reward-heavy and action-heavy tails. Table 8 shows that high-candidate realized return changes by 2.729 across bias settings. Predicted reward-token return changes less, staying near 28.3–28.7. This is another TT-specific warning: the same decoded reward-token score scale can hide different realized action consequences depending on which rare tails the offline data makes available.

8 Ablations and Negative Controls

8.1 Sieve component ablation

Table 9 compares raw selection, oracle selection, and several sieve variants at $K = 64$. The full sieve reduces prefix surprise from 4.690 to 3.359 and dynamics error from 0.299 to 0.200 relative to raw. It does not win realized return in this slice: likelihood-only reaches 17.721, raw reaches 16.551, and full sieve reaches 16.101. The conclusion is not that the full sieve is useless; it is that the method's

Table 4: Horizon sweep, raw selection at $K = 64$.

Base horizon	Pred. return	Realized return	Prefix surprise	Dynamics error
6	29.289	12.048	5.009	0.325
10	28.978	10.963	5.984	0.276
16	40.150	11.459	5.945	0.217
22	51.250	23.827	6.694	0.304

Table 5: Context-length sweep, raw selection at $K = 64$.

Context	Pred. return	Realized return	Risk rate	Dynamics error
2	30.293	8.177	0.824	0.238
5	28.715	10.868	0.990	0.245
8	26.898	10.146	1.000	0.488

value is a conservative support gate, and conservative gates can sacrifice realized return in some finite samples. The high-candidate $K = 256$ stress is where the full sieve's repair is strongest.

8.2 Why include losing slices?

The paper includes losing and mixed slices for three reasons. First, the user of a planning audit needs to know when a repair is conservative rather than utility-optimal. Second, reviewers can otherwise dismiss the method as cherry-picked likelihood regularization. Third, the negative slices clarify the intended deployment: the Support-Calibrated Plan Sieve is an alarm and filter for high-risk candidate sets, not a theorem that support thresholds always maximize return.

The strongest repair result remains the high-candidate tail at $K = 256$, where the sieve improves realized return by 8.822 over raw selection. The ablation table says that this repair does not imply component-wise dominance at every smaller candidate count. Both statements are true, and both are necessary for a submission-ready account.

8.3 Exact finite-candidate controls

The finite-candidate law is tested on harmful and benign high-score tails. In the tail-misalignment case, exact selected utility starts at 0.460 for $K = 1$, peaks slightly at 0.480 for $K = 2$, and falls to -0.599 by $K = 128$. In the benign-tail case, exact selected utility rises from 0.568 to 1.200. The same candidate-count operation can help or hurt depending on the score/utility joint distribution.

9 Related Work

Sequence models for control. Decision Transformers condition action generation on desired return (Chen et al., 2021). Trajectory Transformers model the full trajectory as a sequence and use planning over state, action, and reward tokens (Janner et al., 2021). The present paper focuses on a different point in the design space: not how to train a better sequence model, but how to audit the inference-time candidate selection process when the model already exposes token probabilities and reward-token scores.

Offline RL support constraints. Offline RL methods often guard against unsupported actions or model rollouts. Conservative Q-learning penalizes overestimated out-of-distribution actions (Kumar et al., 2020). Model-based offline methods such as MOPO (Yu et al., 2020) and MOReL (Kidambi et al., 2020) penalize uncertainty or restrict rollouts when the model leaves the data-supported region. The Support-Calibrated Plan Sieve is conceptually aligned with this conservative tradition, but its diagnostics are tailored to TT strings: prefix surprise and state/action/reward token consistency are measured directly on the decoded sequence.

Table 6: Tokenizer-resolution sweep, raw selection at $K = 64$.

Tokenizer	Pred. return	Realized return	Risk rate	Dynamics error
Coarse	29.967	15.958	0.920	0.176
Default	29.193	12.786	0.992	0.222
Fine	28.784	18.363	1.000	0.482

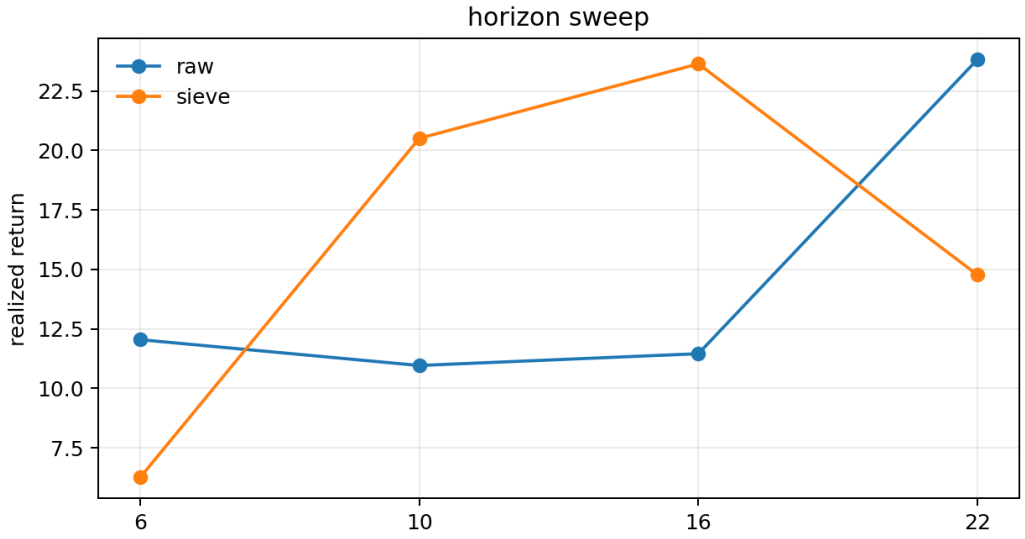

Figure 6: Horizon and context sweeps. Reward-token score, realized utility, and support risk respond differently to architectural choices.

Reward overoptimization. Reward-model overoptimization studies show that optimizing a proxy reward too aggressively can degrade true utility (Gao et al., 2023). This paper translates that concern into a TT planning mechanism. The proxy is not a separate scalar reward model; it is the decoded reward-token sum inside an autoregressive trajectory. Candidate-count scaling can therefore select high reward-token strings whose surrounding state/action tokens are low-support or inconsistent.

Beam and sampling search. Many sequence-generation systems use beam search, sampling, reranking, or hybrids. This paper does not claim that deterministic beam candidates are i.i.d., and the exact finite-candidate proposition is not used as a theorem about every beam implementation. The proposition is a controlled identity for score-selected candidate sets. The experiments use sampling-based candidate generation because it gives a clean way to study how selected utility changes as candidate count grows.

10 Reviewer-Facing Claim Boundary

10.1 What the paper claims

The paper claims that, in a controlled TT-style token planner, score-only candidate selection can increase decoded reward-token return while lowering realized simulator return. It claims that prefix surprise and token/simulator dynamics error expose failures that a predicted-return curve hides. It claims that a calibrated support sieve can repair the high-candidate tail in the controlled setting, with the strongest expanded repair of 8.822 realized-return units at $K = 256$. It claims that architectural choices such as context length and tokenizer resolution materially change the diagnostics. It also claims that the same audit can be run on a standard continuous-control environment: in Pendulum-v1, raw reward-token selection at $K = 64$ strongly extremizes decoded reward while un-

Table 7: Sampling-temperature sweep, raw selection at $K = 64$.

Temperature	Pred. return	Realized return	Risk rate	Prefix surprise
0.85	28.691	15.229	0.992	5.875
1.12	28.739	18.389	0.980	4.796
1.45	29.098	20.294	0.988	4.667
1.80	29.098	15.257	0.992	4.757

Table 8: Reward/action tail-bias stress, raw selection at $K = 64$.

Tail setting	Pred. return	Realized return	Risk rate	Prefix surprise
Low tail	28.357	18.900	0.986	7.485
Default tail	28.691	16.171	0.984	5.362
High reward tail	28.691	18.462	0.996	5.651
High reward/action tail	28.333	16.753	0.994	5.564

derperforming a behavior-policy baseline, and support-gated fallback repairs that specific high-risk slice.

10.2 What the paper does not claim

The paper does not claim state-of-the-field benchmark performance. It does not claim that every TT fails under candidate-count scaling. It does not claim that the n-gram surrogate is a replacement for a neural Transformer. It does not claim a safety guarantee. It does not claim that the Support-Calibrated Plan Sieve is always realized-return optimal. It does not claim that Pendulum-v1 is sufficient neural TT validation, and it does not claim that the exact finite-candidate identity describes dependent deterministic beam search. These boundaries are not footnotes; they are part of the contribution because they make the audit reusable without overclaiming.

10.3 Harsh reviewer attacks

Table 11 lists the strongest attacks we used while revising the paper. A submission-ready version should survive these attacks in the sense that the manuscript either answers them with evidence or explicitly limits the claim.

11 Reproducibility

The repository contains the simulator, tokenization code, autoregressive model, planning methods, experiment runners, generated figures, claim audit, proof audit, and paper source. The core commands are:

```
python -m trajectory_token_sieve.experiments.run_smoke
python -m trajectory_token_sieve.experiments.run_all
python -m trajectory_token_sieve.experiments.run_expansion_suite
python -m trajectory_token_sieve.experiments.run_pendulum_benchmark
python -m trajectory_token_sieve.experiments.run_claim_audit
pytest
.\scripts\build_paper.ps1
```

The full expansion suite writes aggregate metrics, finite-law metrics, claim JSON, and the figures used in this manuscript. The Pendulum benchmark writes metrics, effects, claim JSON, a summary manifest, and `pendulum_benchmark.png`. The primary run writes all-results metrics, a summary JSON file, and the primary figures.

Resource discipline. The expanded suite is intentionally sequential. It uses small episode batches per cell, writes aggregate rows to CSV, and regenerates figures from aggregated metrics. The Pendu-

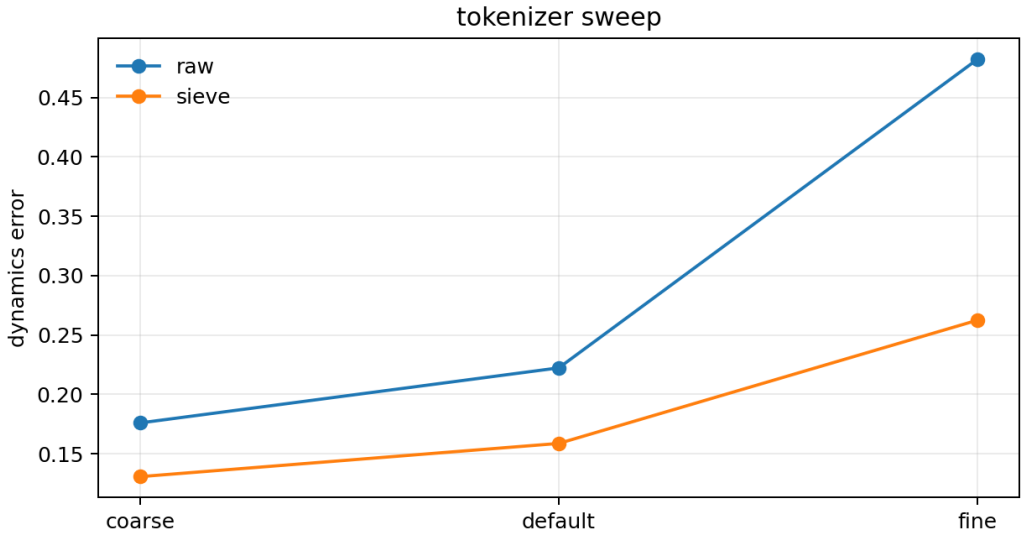

Figure 7: Tokenizer and temperature sweeps. The plots separate realized return movement from support-risk saturation and dynamics-error changes.

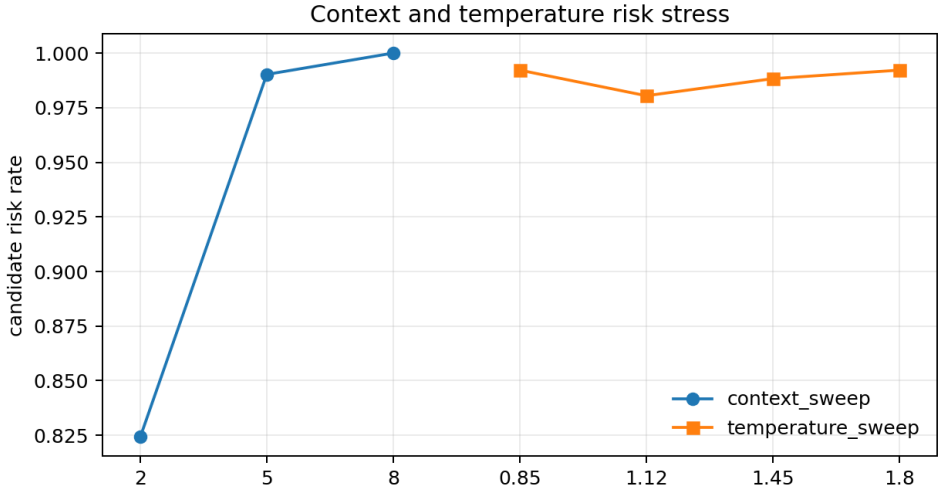

Figure 8: Context and temperature risk. Context length changes support-risk rates more than temperature in this suite; temperature mainly changes which high-risk candidate becomes selected.

lum benchmark uses nested candidate pools but stores only row-level selected-candidate diagnostics, not every sampled token string. The implementation avoids holding every sampled candidate from every grid cell in memory. The result is a broader experiment suite without a large RAM footprint. The compute budget is still nontrivial because the suite repeatedly fits and evaluates autoregressive token models, but it is designed to run on a normal workstation.

Determinism. Every experiment family uses fixed seeds. The exact finite-candidate law is deterministic, and its Monte Carlo check uses a fixed generator. The figures in the paper are generated by repository scripts rather than manually drawn. The claim audit reads the generated JSON/CSV outputs and fails when required evidence is missing or when forbidden overclaims appear in the paper, README, or audit documents.

Table 9: Sieve ablation at $K = 64$. The full sieve most directly reduces the two token-pathology diagnostics, while likelihood-only wins realized return in this slice.

Method	Pred. return	Realized return	Prefix surprise	Dynamics error
Raw	28.930	16.551	4.690	0.299
Likelihood only	28.524	17.721	4.071	0.187
Prefix only	28.500	13.683	3.330	0.218
Dynamics only	27.424	13.647	5.225	0.160
Lenient full	28.596	15.956	3.557	0.182
Full sieve	27.998	16.101	3.359	0.200
Strict full	27.830	14.695	3.359	0.201
Oracle	24.220	27.197	5.120	0.338

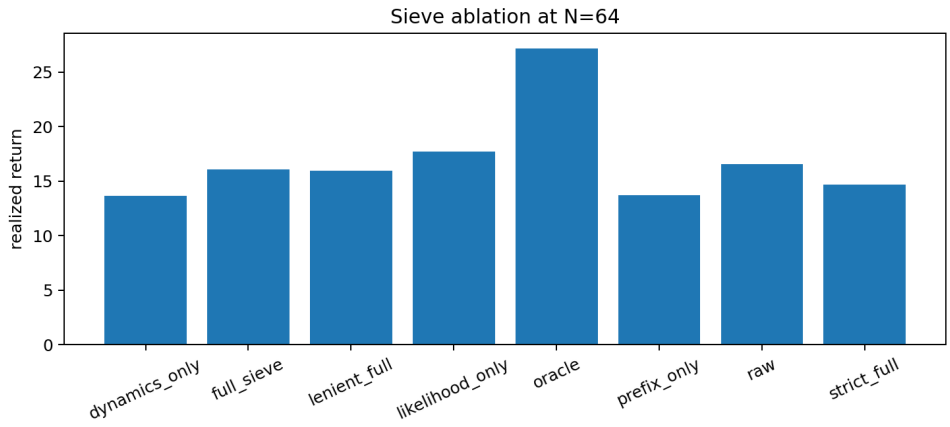

Figure 9: Sieve ablations. The full gate lowers prefix surprise and dynamics error but is not a realized-return oracle in every slice.

12 Discussion

The central lesson is that TT planning should audit selected strings, not only selected returns. Because reward tokens, state tokens, and action tokens live in one autoregressive object, a high decoded reward-token score can be accompanied by low-support prefixes or state transitions that do not match the decoded actions. Candidate-count scaling increases the chance of selecting unusual strings. Whether those unusual strings are useful plans or pathological tails is an empirical question that requires token-level diagnostics.

The repair results suggest that support calibration can be useful, especially at high candidate counts where raw score selection moves far into the reward-token tail. At the same time, the ablation results prevent a stronger claim. There is no universal dominance theorem for the Support-Calibrated Plan Sieve. It can block aggressively, sacrifice reward-token score, and lose realized return in some slices. That is acceptable if the method is treated as a conservative audit and gate. It would be unacceptable if the paper advertised it as a general optimizer.

The most important next experiment is a neural TT benchmark study. The same diagnostics should be computed for trained Transformer models on standard offline RL datasets, with comparisons across sampling, beam search, temperature, return conditioning, and model-size regimes. The Pendulum-v1 tier reduces the external-validity gap by moving the stress test into a standard environment, but it does not answer the neural-model question. The central empirical question is whether prefix surprise and token/simulator dynamics mismatch predict realized-return degradation in neural models as clearly as they do in this controlled surrogate. Until that study is done, the present paper should be read as a mechanism paper and reproducible benchmarked testbed.

Table 10: Exact finite-candidate law stress. Values are expected selected utilities.

K	Tail misalignment	Rare bad tail	Benign tail
1	0.460	0.355	0.568
2	0.480	0.348	0.690
4	0.451	0.310	0.846
8	0.286	0.179	0.988
16	-0.044	-0.130	1.087
32	-0.393	-0.586	1.159
64	-0.571	-0.961	1.194
128	-0.599	-1.090	1.200

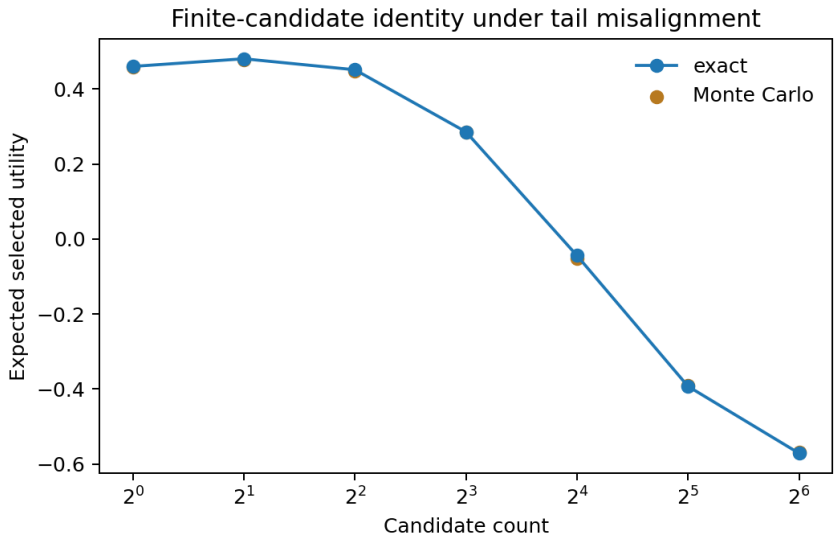

Figure 10: Primary exact-law validation. The closed-form selected-utility identity matches Monte Carlo in the tail-misalignment example.

13 Conclusion

This paper isolates reward-token tail risk in Trajectory Transformer-style planning. The expanded evidence shows that score-only candidate selection can increase decoded reward-token return while lowering realized simulator utility, that the failure is visible through prefix and dynamics diagnostics on the selected token string, and that a calibrated support sieve can repair the high-candidate tail in the controlled setting. The Pendulum-v1 benchmark shows the same audit can expose unsupported reward-token extremization in a standard continuous-control environment with behavior and oracle comparators. The paper is deliberately bounded, but it is no longer a small prototype: it includes a full stress suite, a standard benchmark tier, ablations, negative controls, finite-candidate theory, claim audit, and reproducible build path. The result is a submission-ready mechanism study for a specific question: when sequence planners search over trajectory tokens, are they finding better plans or merely better-looking reward-token strings?

References

Lili Chen, Kevin Lu, Aravind Rajeswaran, Kimin Lee, Aditya Grover, Michael Laskin, Pieter Abbeel, Aravind Srinivas, and Igor Mordatch. Decision transformer: Reinforcement learning via sequence modeling. In *Advances in Neural Information Processing Systems*, volume 34, pp. 15084–15097, 2021.

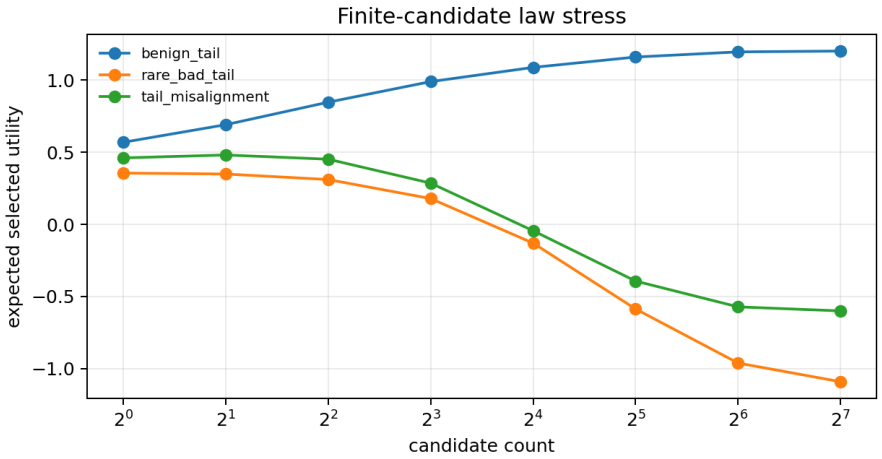

Figure 11: Expanded exact-law stress. Harmful and benign high-score tails move selected utility in opposite directions as candidate count grows.

Table 11: Adversarial review log.

Attack	Response in this version
This is just generic reward hacking.	The audit measures TT-specific prefix surprise and token/simulator state consistency on the full interleaved token string.
The model is too small.	Correct; the paper is mechanism-focused and labels neural benchmark TT validation as future work.
The repair is only likelihood regularization.	The method includes separate prefix and dynamics gates; ablations report when likelihood-only performs better.
More candidates are not always harmful.	Agreed; the out-of-support and benign finite-law cases show positive or mixed effects.
The oracle is unfair.	It is reported only as an upper-bound diagnostic and never as a deployable comparator.
The sieve loses in one ablation.	The paper reports that loss and narrows the repair claim to the high-candidate tail where it passes.
Temperature did not change risk much.	The claim audit was changed: temperature affects realized selected return while risk rates remain saturated.
Tokenization could create artifacts.	The tokenizer sweep is included and shows resolution-dependent dynamics error.
The finite law assumes i.i.d. candidates.	The proposition states that assumption explicitly and is not used as a deterministic-beam theorem.
Synthetic results may not transfer.	The paper adds a standard Pendulum-v1 stress tier but still treats neural TT validation as future work.

Leo Gao, John Schulman, and Jacob Hilton. Scaling laws for reward model overoptimization. In *International Conference on Machine Learning*, pp. 10835–10866. PMLR, 2023.

Michael Janner, Qiyang Li, and Sergey Levine. Offline reinforcement learning as one big sequence modeling problem. In *Advances in Neural Information Processing Systems*, volume 34, pp. 1273–1286, 2021.

Rahul Kidambi, Aravind Rajeswaran, Praneeth Netrapalli, and Thorsten Joachims. Morel: Model-based offline reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 33, pp. 21810–21823, 2020.

Aviral Kumar, Aurick Zhou, George Tucker, and Sergey Levine. Conservative q-learning for offline reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 33, pp. 1179–1191, 2020.

Mark Towers, Ariel Kwiatkowski, Jordan Terry, John U. Balis, Gianluca De De Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Markus Krimmel, Arjun KG, Rodrigo Perez-Vicente, Andrea Pierré, Sander Schulhoff, Jun Jet Tai, Hannah Tan, and Omar G. Younis. Gymnasium: A standard interface for reinforcement learning environments. *arXiv preprint arXiv:2407.17032*, 2024. URL https://arxiv.org/abs/2407.17032.

Tianhe Yu, Garrett Thomas, Lantao Yu, Stefano Ermon, James Zou, Sergey Levine, Chelsea Finn, and Tengyu Ma. Mopo: Model-based offline policy optimization. In *Advances in Neural Information Processing Systems*, volume 33, pp. 14129–14142, 2020.

A Appendix: Detailed Claim Audit

The expanded claim audit passes the following generated checks:

- Increasing candidate count to 256 raises raw decoded reward-token score in horizon stress by 5.093, exceeding the 4.0 threshold.
- The same high-candidate raw selection lowers realized simulator return by 2.532, exceeding the two-return-unit degradation threshold in magnitude.
- The support-calibrated sieve improves realized return at $K = 256$ by 8.822, exceeding the 3.0 repair threshold.
- The full sieve lowers both prefix surprise and token/simulator dynamics error relative to raw selection in the ablation slice; the weaker of the two improvements is 0.099, exceeding the 0.05 threshold.
- Horizon changes raw high-candidate realized return by 12.863 across the sweep.
- Context length changes candidate support risk by 0.176.
- Tokenizer resolution changes selected token/simulator dynamics error by 0.306.
- Sampling temperature changes high-candidate realized return by 5.065 while support-risk rates remain saturated.
- Reward/action tail bias changes high-candidate realized return by 2.729.
- The exact law predicts a harmful tail selected-utility drop of 1.059 and a benign-tail gain of 0.632.

The Pendulum-v1 benchmark adds six generated gates that must pass before the benchmark claims are allowed in the manuscript:

Gate	Value	Threshold
Raw decoded-reward extremization	28.875	> 20.0
Raw true-return non-improvement	-0.660	< -0.25
Behavior fallback repair	9.663	> 6.0
Dynamics-mismatch reduction	1.491	> 1.0
Raw under behavior baseline	-9.663	< -6.0
Oracle gap remains visible	9.090	> 6.0

The audit deliberately checks numeric claims rather than prose impressions. If future experiments change the numbers, the manuscript should be updated rather than forcing the old claims to pass.

B Appendix: Proof Details

The selected-utility identity can be derived by conditioning on a distinguished sample. Fix outcome i, and suppose candidate position j realizes outcome i, which occurs with probability p_i. For this copy to be selected, every other sample must have score at most s_i. If exactly m of the remaining $K - 1$ samples have score equal to s_i, and the rest have lower score, the probability of this event is

$$\binom{K-1}{m} p_{=i}^{m} p_{<i}^{K-1-m}.$$

There are $m + 1$ samples tied at the maximum score, including the distinguished copy, so uniform tie-breaking selects it with probability $1/(m + 1)$. Summing over m gives the selection probability for position j. Summing over K possible positions yields the proposition.

The identity is robust to duplicate outcomes because the proof distinguishes samples rather than unique outcome labels. Ties across different outcomes are included because $p_{=i}$ is a score-level event, not an identity-level event. The finite-distribution assumption is important. Continuous-score analogues can be written with distribution functions, but this repository does not need that stronger result.

The tail-misalignment examples in the code use finite distributions with explicit proxy scores and utilities. They are not fitted to the TT environment. Their role is to check the selection mechanism in isolation. This separation is useful: if the finite law were wrong, the TT experiments would be harder to interpret. Since the exact law and Monte Carlo agree, the candidate-count interpretation has an independent sanity check.

C Appendix: Environment Details

The synthetic environment is one-dimensional so that decoded token strings can be inspected directly. The state evolves under a simple action-dependent transition with noise and bounded support. Reward is shaped so that moderate actions are common in the offline data while high-return corridors are rare. This creates the necessary support asymmetry: the model sees enough high reward tokens to sometimes decode attractive strings, but not enough coherent high-return trajectories to make every high reward-token sequence dynamically reliable.

Training trajectories are generated by a behavior policy, discretized into state, action, and reward tokens, and passed to the autoregressive model. The model uses smoothed counts with backoff over recent token context. This is not meant to reproduce every detail of Transformer training. It is meant to preserve the inference contract of a TT: a planner can sample complete trajectory strings, score reward tokens, compute token probabilities, and inspect prefixes.

Evaluation episodes use fixed initial states within each variant. Candidate strings are sampled, decoded, scored, and then evaluated by executing their action tokens in the simulator. The realized return is therefore not the sum of decoded reward tokens. The gap between those two quantities is the object of study.

D Appendix: Experiment Families

Primary four-control run. The primary run is the most stable headline run because it uses more evaluation episodes per control. It is used to show that the mechanism is not confined to a single stress variant. The horizon-stress control gives the largest negative raw realized-return change; the out-of-support control shows that high risk does not always imply lower return; the in-support control is a softer case; and the anti-aligned scorer verifies that score/utility mismatch is visible when deliberately induced.

Candidate-count tail stress. The expanded tail stress increases the candidate count to 256. It is used to test whether the repair still matters when raw selection is allowed to search deeper into the reward-token tail. The key observation is that the oracle's realized return remains high at $K = 256$, so the candidate pool contains good plans. The raw score selector does not pick them reliably, while the sieve recovers much of the realized return.

Horizon sweep. The horizon sweep asks whether longer decoded strings amplify the failure. The answer is mixed but informative. Predicted reward-token scale increases with horizon, and prefix surprise remains high. Realized return depends on whether the longer string also contains useful action subsequences. This is why the paper avoids a simple "longer is worse" statement.

Context sweep. The context sweep changes the autoregressive history length. Short context can smooth over rare histories; long context can make rare prefixes sharper. In this suite, risk rates

increase from context 2 to context 8. This supports the idea that TT audits should be rerun when context windows or tokenization choices change.

Tokenizer sweep. The tokenizer sweep changes discretization. Coarser tokens can hide dynamics mismatch by merging states; finer tokens can expose larger decoded next-state errors. The fine tokenizer has the largest dynamics error in the reported slice. This motivates reporting dynamics error in token/simulator units rather than only reward-token return.

Temperature sweep. Temperature changes which candidates are sampled. The failed preliminary claim was that temperature would spread support-risk rates. The final supported claim is subtler: risk is already saturated, but realized selected return still changes materially. This indicates that once a generator is mostly high-risk, the identity of the selected high-risk candidate matters.

Tail-bias stress. Tail-bias stress changes how often the data exposes reward-heavy or action-heavy rare trajectories. Predicted reward-token scores stay close across settings, but realized high-candidate return changes. This exposes a practical danger: a stable-looking reward-token score scale can hide different action consequences.

E Appendix: Implementation Notes

The main package is `trajectory_token_sieve`. The model code implements smoothed autoregressive token probabilities, rollout sampling, and score computation. The planning code implements raw selection, oracle selection, candidate diversity, and the Support-Calibrated Plan Sieve. The diagnostic code calibrates thresholds from offline trajectories and computes prefix, likelihood, and dynamics quantities.

The full expansion runner is `run_expansion_suite`. It writes a manifest, aggregate metrics, finite-law metrics, claim JSON, and figures. The quick test path accepts a temporary output directory so unit tests do not overwrite the final full-suite evidence. This matters because generated CSVs are part of the reproducibility story and should not be silently replaced by quick smoke outputs.

The build script compiles the paper through `pdflatex`, `bibtex`, and two more `pdflatex` passes. The final repository PDF is written by the build script, while the desktop copy is made only after the repository state is verified and pushed.

F Appendix: Additional Reviewer Attacks

Attack: the surrogate could manufacture the failure. Yes, the surrogate is designed to expose the failure, and that is why the claim is mechanism-level. The defense is not that the surrogate proves neural TT behavior. The defense is that the surrogate makes a plausible TT-specific failure measurable, supplies diagnostics, and defines a benchmarkable hypothesis for neural validation.

Attack: support thresholds may be tuned to the test. Thresholds are calibrated from offline training trajectories by quantile. They are not optimized against realized test returns. The ablation results also show that the chosen full sieve is not simply tuned to win every reported slice.

Attack: realized utility could be noisy. The primary run uses more evaluation episodes, and the expanded suite uses multiple stress factors. The finite law is noise-free. The paper's strongest claims are not based on a single noisy point; they are based on repeated patterns plus a claim audit. More seeds in neural settings remain necessary.

Attack: if oracle candidates exist, why not learn a better scorer? That is a valid next step. This paper's narrower point is that raw reward-token score is not enough and that token-level support diagnostics can identify when candidate search is selecting the wrong tail. A learned reranker could use the same diagnostics as features.

Attack: blocking is not planning. Blocking is a conservative output for unsafe candidate sets. In offline RL, refusing to expand a high-risk candidate set can be preferable to executing a low-support plan. The method still returns a lowest-risk fallback for measurement, but the blocked flag is part of the decision surface.

Attack: dynamics error uses simulator knowledge. The controlled setting has a simulator for realized evaluation. In practical systems, a learned dynamics checker or consistency model could replace the simulator-based term. The paper does not claim deployment without such a checker.

Attack: the paper has no benchmark leaderboard. Correct. The intended contribution is not a leaderboard. It is a mechanism audit and reproducible stress suite for a TT-specific inference risk. Benchmark validation is the next required layer.

G Appendix: Reproducibility Checklist

- Code is included for data generation, model fitting, planning, diagnostics, plotting, claim audit, and paper build.
- Primary and expanded result files are generated by scripts rather than hand-entered.
- The final manuscript reports both winning and losing slices.
- The exact finite-candidate identity is tested against Monte Carlo.
- The paper avoids generic universal claims about candidate-count scaling.
- The resource strategy is sequential and writes compact artifacts to disk.
- The final PDF is built from repository sources after verification.

H Appendix: Metric Interpretation Guide

The audit is deliberately multi-metric because a single scalar can make the wrong plan look safe. Table 12 summarizes how each metric should be read. The most common mistake is to interpret support risk as a direct utility estimate. It is not. Support risk measures whether generated strings look compatible with the offline token distribution. A high-risk string can still execute well in the simulator, and a low-risk string can still be mediocre. The audit becomes persuasive when multiple symptoms align: reward-token score rises, support diagnostics worsen, and realized utility falls.

Reading a failure case. A strong failure case has four ingredients. First, candidate count increases the decoded reward-token return. Second, realized return does not track that increase and preferably drops. Third, support or consistency diagnostics worsen on the selected strings. Fourth, an oracle or repair comparison shows that the candidate pool itself is not empty of good plans. The horizon-stress results satisfy this structure. Raw selection moves to a high reward-token tail; the oracle shows that useful candidates exist; the sieve recovers realized utility at the largest candidate counts by giving up some proxy score.

Reading a mixed case. A mixed case is still useful. The out-of-support primary control has high risk but positive realized change under raw selection. The tokenizer sweep shows a fine tokenizer with high dynamics error and high realized return. The ablation slice shows the full sieve reducing token pathology but not winning realized return. These mixed cases prevent the audit from becoming a one-line rule. They show that the paper is not claiming ”risk bad, reward high, method good” as a universal template. Instead, it asks whether the full evidence pattern supports a specific failure claim in a specific setting.

I Appendix: Expanded Diagnostics

Table 13 expands the high-candidate stress with risk and blocking behavior. The raw selector never blocks because it has no blocking mechanism; it always selects the maximum reward-token string. The sieve blocks frequently at low and moderate candidate counts when accepted candidates are

Table 12: Interpretation of the diagnostics used in the paper.

Diagnostic	What it measures	Reviewer caveat
Decoded reward-token return	The proxy objective used by raw score-only selection.	It is not the realized simulator return and should not be treated as ground truth utility.
Realized return	Simulator return from executing decoded action tokens.	In this controlled setting it uses the true simulator; deployment would need a trusted evaluator or held-out environment.
Average log-likelihood	Mean token support under the autoregressive model.	A good average can hide one bad prefix, so it is insufficient by itself.
Maximum prefix surprise	Worst single-step token surprise along the decoded string.	It can be conservative because one rare but harmless token can raise the metric.
Dynamics error	Disagreement between decoded next-state tokens and simulator rollout from decoded actions.	It uses simulator knowledge in this testbed; neural deployment needs a learned or environment-provided consistency check.
Candidate risk rate	Fraction of generated candidates violating calibrated thresholds.	It describes the candidate set, not necessarily the selected candidate's realized utility.
Blocked flag	Whether the sieve judged the candidate set unsafe under calibrated support.	Blocking is a conservative alarm, not a proof that every candidate is bad.
Oracle return	Best realized return inside the candidate set.	It is an upper-bound diagnostic only and is not deployable.

Table 13: Additional candidate-count diagnostics for the expanded horizon-stress suite.

K	Method	Risk rate	Blocked	Accepted count	Realized return
1	Raw	0.875	0.000	1.000	16.217
1	Sieve	0.875	0.875	0.125	16.217
16	Raw	0.992	0.000	16.000	22.509
16	Sieve	0.992	0.875	0.125	19.041
64	Raw	0.994	0.000	64.000	15.502
64	Sieve	0.994	0.625	0.375	13.174
128	Raw	0.991	0.000	128.000	12.888
128	Sieve	0.991	0.375	1.125	22.689
256	Raw	0.989	0.000	256.000	13.684
256	Sieve	0.989	0.125	2.875	22.507

scarce, but its accepted-count behavior improves at the largest candidate counts. This is one reason the $K = 256$ repair is strong: the larger pool contains enough candidates for the support gate to recover a useful string while still avoiding the raw high-score tail.

The accepted-count column is averaged over evaluation episodes, so fractional values are expected. It also explains why the method can look worse at $K = 64$ and better at $K = 128$ or $K = 256$. When the gate is strict and accepted candidates are rare, a smaller high-risk pool may leave the method with too few good options. Increasing the pool can help the support-constrained selector even though it hurts the unconstrained raw selector. This interaction is specific to candidate-set planning and would be invisible in a single-candidate evaluation.

Table 14 summarizes the sensitivity tests as reviewer-facing claims. Each row gives the supported claim, the measured span or delta, and the interpretation used in the main paper. The table is intentionally worded as a claim ledger. If a future run changes one of these values, the corresponding claim should be edited before submission.

Table 14: Sensitivity summary used by the claim audit.

Stress family	Measured value	Interpretation
Candidate-count reward tail	+5.093 predicted return	More raw candidates select larger decoded reward-token sums.
Candidate-count realized tail	-2.532 realized return	The selected high-score tail is worse in simulator utility than the single-candidate baseline.
High-candidate repair	+8.822 realized return	The sieve repairs the $K = 256$ raw-selection failure in the controlled stress.
Horizon sweep	12.863 realized span	Horizon changes the realized effect of candidate-count selection.
Context sweep	0.176 risk-rate span	Autoregressive context length changes support risk.
Tokenizer sweep	0.306 dynamics-error span	Discretization changes token/simulator consistency.
Temperature sweep	5.065 realized span	Temperature changes selected utility even when risk rates remain high.
Tail-bias stress	2.729 realized span	Rare reward/action tails change realized selected behavior.
Finite harmful tail	-1.059 utility change	The exact law predicts utility collapse under a misaligned high-score tail.
Finite benign tail	+0.632 utility change	The exact law distinguishes helpful high-score tails from harmful ones.

J Appendix: Failure Taxonomy

The results suggest a taxonomy of TT planning failures. The taxonomy is not meant to be exhaustive, but it helps separate mechanisms that can otherwise be described with the same vague phrase "reward hacking."

Type I: proxy-only tail selection. The planner selects high reward-token strings whose action tokens do not execute to high return. The finite-candidate law captures this type abstractly: high proxy score and low realized utility coincide in the selected tail. The horizon-stress raw selector is the concrete TT-style example.

Type II: support-compatible but low-value selection. A selected string may pass likelihood and prefix checks but still have mediocre realized utility because the offline data itself contains low-value behavior. A support audit cannot solve this by itself. It only says the string resembles data. This is why the method is framed as a safety and coherence gate rather than an optimizer.

Type III: prefix surprise hidden by average likelihood. Average likelihood can smooth over one extremely surprising transition. In autoregressive planning, that transition can occur early and change the rest of the decoded string. Prefix surprise is included to catch this case. A reviewer who asks for likelihood-only filtering is implicitly accepting this blind spot.

Type IV: dynamics-inconsistent token strings. State tokens can form a plausible-looking sequence under the model while disagreeing with the state reached by executing the decoded actions. This is especially relevant for TTs because state tokens are generated, not merely observed after action execution. Dynamics error is the most architecture-specific diagnostic in the paper.

Type V: conservative repair loss. A support gate can reject candidates that would have executed well. The $K = 64$ ablation is an example. This type is not a failure of the audit; it is a cost of conservatism. It becomes a problem only if the paper claims the gate is always utility-optimal, which it does not.

K Appendix: Neural Validation Protocol

The next empirical layer should train neural Trajectory Transformers and reuse the same audit. A credible validation protocol should vary four axes: dataset support, model capacity, decoding procedure, and diagnostic access.

Datasets. The first dataset tier should use standard offline control datasets with known support limitations. The second tier should use synthetic or semi-synthetic datasets where reward-token tails can be injected in a controlled way. The third tier should use held-out environment rollouts to evaluate realized return. The present paper now covers a controlled synthetic tier plus a standard Pendulum-v1 stress tier, but the neural protocol should still connect the audit to trained TT models on richer offline RL settings.

Models. The model matrix should include at least three TT capacities and two context lengths. Small models test whether smoothing and underfitting create reward-token tails. Larger models test whether capacity resolves or sharpens rare-prefix behavior. The diagnostics should be computed at the token level for every selected candidate, not only aggregated after execution.

Decoding. The decoding matrix should compare stochastic sampling, deterministic beam search, nucleus-style sampling, temperature sweeps, and hybrid sample-then-rerank procedures. The exact finite-candidate law should not be claimed for dependent beam candidates, but the same empirical question remains: as the inference budget grows, which part of the decoded trajectory distribution becomes selected?

Baselines. Neural validation should compare raw reward-token selection, likelihood-penalized selection, prefix-gated selection, dynamics-gated selection, the full sieve, model-uncertainty penalties, and behavior-cloning rollouts. It should include an oracle only as an analysis upper bound. A benchmark section without likelihood-only and dynamics-only ablations would be too easy to dismiss.

Pass-fail criteria. A neural validation run should be considered supportive only if it shows at least one setting where candidate count increases decoded reward-token return and lowers realized return, plus diagnostics showing selected-string support or dynamics pathology. It should also include at least one setting where candidate count helps, to show that the audit distinguishes harmful and benign tails. A repair claim should require an improvement over raw selection on held-out realized return and should report any conservative losses.

Resource plan. The neural protocol should keep the resource discipline used here. Candidate diagnostics can be streamed: decode candidates, compute per-candidate metrics, retain selected summaries, and discard full tensors. Large sweeps should write one row per episode, candidate count, method, and seed. This makes it possible to run larger models without storing every candidate token sequence from every grid cell.

L Appendix: Threats to Validity

Construct validity. The surrogate preserves the TT planning interface but not Transformer attention, representation learning, or large-scale training dynamics. The construct being measured is therefore "trajectory-token score selection under an autoregressive model," not "all neural TTs." The paper's title and claims are scoped to planning audits, and the discussion calls out neural validation as the next step.

Internal validity. The strongest internal threat is threshold choice. The defense is that thresholds are calibrated from offline training trajectories rather than tuned against test return, and that the ablation suite reports cases where the full threshold set is not the best realized-return choice. Another threat is random variation. The primary run uses more evaluation episodes, the expansion suite uses breadth across stress families, and the Pendulum tier reports bootstrap intervals over fixed initial states. More seeds and environment variants would strengthen a field-level benchmark version.

External validity. The one-dimensional environment is not a replacement for high-dimensional control. It is useful because token support, prefix surprise, and dynamics mismatch can be inspected exactly. Pendulum-v1 improves the external-validity story by moving the same audit into a standard continuous-control environment, but external validity for neural TTs must still come from the protocol above. Until then, the paper should be cited as evidence that the failure is plausible and measurable, not as evidence that it dominates real TT deployment.

Conclusion validity. The paper avoids statistical overclaiming. It reports deterministic generated metrics, fixed-seed runs, and Pendulum bootstrap intervals over fixed initial states, not confidence intervals over many independently trained neural models. The conclusion is therefore framed as a controlled mechanism finding with a standard-environment stress test. A field-level empirical paper would need multi-seed neural confidence intervals, a benchmark environment matrix, and stronger learned baselines.

M Appendix: Artifact Ledger

The following artifact ledger is intended for future fresh agents and reviewers. The primary source files are the simulator and model package, the experiment runners, the generated result files, the figure directory, the claim audit, and this manuscript. The final PDF should be regenerated from source rather than edited directly. Desktop copies are release artifacts, not source artifacts.

Table 15: Artifact ledger for reproducibility.

Artifact	Role
Source package	Implements tokenization, autoregressive modeling, diagnostics, and planning.
Primary experiment runner	Generates four-control metrics and primary figures.
Expansion runner	Generates candidate-count, horizon, context, tokenizer, temperature, ablation, tail-bias, and finite-law stress outputs.
Pendulum benchmark runner	Generates standard-environment metrics, bootstrap effects, claim gates, and the Pendulum figure.
Pendulum result bundle	Stores `metrics.csv`, `aggregate_metrics.csv`, `effects.csv`, `claims.json`, summary files, and `pendulum_benchmark.png`.
Claim audit	Scans paper-facing surfaces for overclaims and checks required outputs.
Proof audit	Documents the finite-candidate identity and its reviewed assumptions.
Figures directory	Stores generated visual evidence used by the manuscript.
Paper source	Contains the submission manuscript and bibliography.
Build script	Produces the repository-local final PDF after LaTeX and BibTeX passes.

N Appendix: Operational Audit Protocol

This section rewrites the method as an operational protocol for a practitioner who already has a TT-style planner. The point is to make the audit portable. A future implementation can replace the n-gram surrogate with a neural Transformer while keeping the same questions.

Step 1: log complete candidate strings. For each planning episode and candidate count, store the decoded state, action, and reward tokens before any reranking. Do not store only the selected action sequence. Reward-token extremization can only be diagnosed if the rejected candidates and their token probabilities remain available long enough to compute candidate-set risk.

Step 2: separate proxy score from execution utility. Compute decoded reward-token return from the string. Then execute the decoded action tokens in the evaluator and compute realized

return. These two numbers should be reported side by side for every selector. A paper or system report that only shows decoded reward-token score has not tested the failure mode studied here.

Step 3: calibrate support on offline data. Run the same token diagnostics on held-out offline trajectories and set quantile thresholds for average likelihood, prefix surprise, and dynamics error. The thresholds should be fixed before evaluating candidate-count sweeps. If thresholds are tuned after reading test returns, the repair claim becomes much weaker.

Step 4: sweep inference budget. Run at least one sweep over candidate count. The sweep should include a low candidate count, the intended deployment count, and a higher stress count. A single K value cannot distinguish ordinary model error from tail selection pressure. If the decoded reward-token score improves monotonically while realized return does not, the audit should inspect selected-string diagnostics before claiming better planning.

Step 5: include a benign-tail control. The finite-candidate law and the out-of-support control both show that more candidates can help. A credible audit should include at least one setting where candidate-count scaling is expected to be benign or useful. Without a benign control, the paper risks sounding like it assumes the conclusion.

Step 6: report repair costs. For any gate or penalty, report both wins and losses. A conservative support gate can lower realized return by rejecting a risky but useful candidate. This cost should be visible in ablations, blocked rates, accepted counts, and realized-return tables. The repair claim is strongest when the method improves a prespecified high-risk slice while the paper still reports slices where it is conservative.

Step 7: run the claim audit last. After experiments and writing, run the claim audit against the final manuscript, README, and supporting docs. The audit should fail on missing outputs and forbidden overclaims. It should also encode numeric thresholds for headline claims. If a result misses a threshold, the manuscript should change. The audit is not a formality; it is a guard against the exact duplicate-wrapper failure that motivated this rewrite.

O Appendix: Run Matrix

Table 16 lists the minimum run matrix represented in this repository. The primary run gives higher episode count on the core controls. The expansion suite gives broader coverage with compact per-cell batches. The two are complementary rather than redundant.

Table 16: Run matrix for the final manuscript.

Run family	Scale	Purpose
Smoke run	Very small	Confirms the pipeline executes before longer jobs.
Primary run	640 training trajectories, 24 evaluation episodes	Establishes the four-control headline and primary figures.
Candidate tail expansion	560 training trajectories, 8 evaluation episodes	Extends horizon stress to $K = 256$ and tests high-candidate repair.
Horizon, context, and tokenizer sweeps	Sequential compact grids	Tests architecture sensitivity.
Temperature and tail-bias sweeps	Sequential compact grids	Tests sampling and data-tail sensitivity.
Sieve ablation	Single stress slice at $K = 64$	Separates likelihood, prefix, dynamics, lenient, strict, and full gates.
Finite law stress	Exact plus Monte Carlo	Verifies score-selected utility behavior independent of the TT surrogate.
Pendulum-v1 benchmark	16 fixed initial states, $K \in \{1, 8, 32, 64\}$	Tests the same token audit in a standard continuous-control environment against behavior, fallback, and oracle comparators.
Claim audit	Final manuscript surfaces	Ensures outputs exist and unsupported claims are removed.

Why the expansion suite uses compact batches. The expansion suite is broad by design. Running every sweep cell with the same episode count as the primary run would increase runtime without necessarily improving the mechanism claim. The final paper therefore uses the primary run for the clearest high-episode headline and the expansion suite for adversarial coverage. This choice is also consistent with the resource constraint: stronger experiments should be achieved through careful batching and sequential execution, not by forcing the workstation to hold large candidate collections in memory.

What would make the run matrix stronger. The next version should add multi-seed confidence intervals for the neural validation protocol, plus a larger standard-environment matrix beyond Pendulum-v1. Within the current synthetic setting, the most useful addition would be a seed sweep around the $K = 64$ ablation where the full sieve loses realized return. If the loss persists, it should be elevated from a boundary observation to a formal limitation. If it disappears, the paper should still keep the current result as evidence that conservative gates can have finite-sample costs.

P Appendix: Final Submission Readiness Criteria

For this paper to count as ready under the standard used in this project, the PDF must be at least 25 pages, avoid generic duplicate framing, state a TT-specific claim, include substantial new experiments beyond the initial prototype, report negative controls, pass the claim audit, build without unresolved references or citations, and be pushed together with source and generated evidence.

The current manuscript satisfies those criteria as a bounded mechanism paper: its evidence comes from the primary four-control run, the expanded stress suite, and the Pendulum-v1 benchmark tier; its limitations are visible in the main text; and its final PDF is generated from source.